ZimaBlade 7700 Cluster - Ubuntu Server Setup Guide

Chrony NTP | AIC8800 Wi-Fi | DKMS | Tailscale & SSH

November 6, 2026

Purpose and environment

This runbook is for three ZimaBlade 7700 computers with Ubuntu Server, UGREEN AIC8800-based USB Wi-Fi adapters, and a controller/two-worker distributed C++ cluster. It captures the successful **ronnyf manual driver installation** while explaining a separate **DKMS package** approach, time synchronization with **Chrony**, and private remote access through **Tailscale**.

Node	Suggested hostname	Role	Repeat these sections
Node 1	zimablade-controller	C++ task dispatcher	Chrony, Wi-Fi, optional DKMS, Tailscale
Node 2	zimablade-worker1	C++ worker	Chrony, Wi-Fi, optional DKMS, Tailscale
Node 3	zimablade-worker2	C++ worker	Chrony, Wi-Fi, optional DKMS, Tailscale

Important ground rules

- Run commands **on each ZimaBlade separately**, unless labeled as a command for your Windows laptop.
- Keep a local keyboard and monitor available until reboot tests pass. Do not replace Wi-Fi modules over the only working remote connection.
- **Choose one driver installation method per node**: Section 3 (manual make install) *or* Section 4 (ronnyf DKMS .deb). Mixing them can produce duplicate module files and DKMS warnings.
- The driver source revision matters; a working module on kernel 7.0.0-30-generic does **not** guarantee it will build on every later kernel.
- These instructions are a reproducible reference, not a record of actions executed on the other nodes. Confirm the actual kernel and device ID on each node.

1. Preflight: identify Ubuntu, kernel, USB hardware, and network

Start each node with a **temporary internet connection** (Ethernet or Android USB tethering) and run:

```
hostnamectl
cat /etc/os-release
uname -r
lsusb
ip link
nmcli device status
```

Confirm the target adapter is visible in lsusb; for the AIC8800 device previously seen on the controller, the ID was a69c:8d80 (aicsemi / AIC Wlan). Other USB IDs or adapters may require different drivers. lsusb visibility only confirms USB detection, not a functioning wireless interface.

Check IP connectivity and DNS separately:

```
ping -c 4 1.1.1.1
getent hosts github.com
```

If Ubuntu cannot resolve domains or HTTPS reports certificates “not yet valid,” verify the clock and complete **Section 2** before installing drivers or Tailscale.

2. Chrony: automatic time synchronization (NTP)

2.1 Fix the time zone and initial clock

Ubuntu can keep hardware/UTC time in UTC while displaying Eastern local time. The preferred time zone for your location is America/New_York; DST changes are handled automatically.

```
sudo timedatectl set-timezone America/New_York
timedatectl status
```

If the clock is *months* wrong, set a reasonably accurate current **local** time first, replacing the example date/time with the actual current value:

```
sudo timedatectl set-ntp false
sudo timedatectl set-time "YYYY-MM-DD HH:MM:SS"
```

Do **not** type the example placeholders literally. On Ubuntu 26.04, systemd-timesyncd.service may not be installed; Chrony is Ubuntu’s default time daemon for new installations.

2.2 Handle the APT lock safely

Do not run two package-manager operations simultaneously. If apt reports that unattended-upgr owns /var/lib/dpkg/lock-frontend, first inspect the process rather than deleting lock files:

```
ps -ef | grep '[u]nattended-upgrade'
sudo systemctl status apt-daily-upgrade.service
sudo tail -n 30 /var/log/unattended-upgrades/unattended-upgrades.log
```

Let normal automatic updates finish if they are progressing. If clearly stuck, diagnose/stop the responsible service carefully from local console; do **not** remove /var/lib/dpkg/lock* by hand or repeatedly kill dpkg. Once all package-manager operations have exited, recover any incomplete configuration:

```
sudo dpkg --configure -a
sudo apt update
```

2.3 Install, start, and verify Chrony

```
sudo apt install -y chrony
sudo systemctl enable --now chrony
systemctl is-active chrony
chronyc tracking
chronyc sources -v
timedatectl status
```

Healthy outputs: chrony is active; chronyc tracking reports Leap status : Normal; chronyc sources -v contains a selected source marked ^*; and timedatectl status eventually reports System clock synchronized: yes and NTP service: active.

If the initial offset is still large, a one-time correction can be requested **before launching workloads**:

```
sudo chronyc makestep
chronyc waitsync 30 0.1
chronyc tracking
```

makestep can jump the clock. Later Chrony normally slews it gradually. For C++ task durations, use std::chrono::steady_clock rather than wall-clock timestamps.

NTP connectivity: Ubuntu Chrony may use NTS; that can require outbound UDP 123 and TCP 4460. If chronyc sources -v never finds a usable source, check DNS, your clock, and outbound firewall rules. You do not need to configure any worker as an NTP **server** just to synchronize its own clock.

3. AIC8800 Wi-Fi: ronnyf manual build (your proven method)

This is the method that successfully worked on one node. It installs the driver for the **currently running kernel**; it does **not** register the driver with DKMS automatically.

3.1 Check chipset and prerequisites

Connect the UGREEN adapter and verify its identity:

```
lsusb
sudo dmesg | tail -n 40
uname -r
```

Install build dependencies over the temporary internet connection:

```
sudo apt update
sudo apt install -y git build-essential gcc \
 linux-headers-$(uname -r) network-manager iw usbutils eject
```

If linux-headers-$(uname -r) cannot be found, resolve that kernel/header mismatch before compiling. NetworkManager must actually manage the wireless interface; Ubuntu Server installations may instead use Netplan plus systemd-networkd.

3.2 Clone, compile, and verify modules

```
cd ~
git clone https://github.com/ronnyf/AIC8800-Linux-Driver.git
cd AIC8800-Linux-Driver
git rev-parse --short HEAD
make -C drivers/aic8800 CC=gcc
find drivers/aic8800 -name '*.ko'
```

Before proceeding, confirm compilation **exits successfully** and both aic_load_fw.ko and aic8800_fdrv.ko exist. Save the Git commit hash in your setup notes to reproduce the same driver source on other nodes. The repository also documents an LLVM/Clang build (make LLVM=1 -C drivers/aic8800); the GCC command above matches your successful troubleshooting route. Do not switch toolchains merely because a make step is slow.

3.3 Install firmware and modules, then load

```
sudo make -C drivers/aic8800 install
sudo depmod -a
sudo modprobe aic_load_fw
sudo modprobe aic8800_fdrv
lsmod | grep aic
modinfo -n aic8800_fdrv
```

The repository's install Make target copies firmware, udev rules, and .ko files. It does **not** register DKMS. If modprobe fails, inspect sudo dmesg | tail -n 60 before changing anything.

3.4 Connect to Wi-Fi and enable autoconnect

```
nmcli device status
nmcli device wifi list
```

Replace YOUR_INTERFACE (example: wlx...) and YOUR_SSID with the values shown on **this** ZimaBlade:

```
sudo nmcli device set YOUR_INTERFACE managed yes
sudo nmcli --ask device wifi connect "YOUR_SSID" \
  ifname YOUR_INTERFACE
nmcli -f NAME,DEVICE connection show --active
sudo nmcli connection modify "YOUR_CONNECTION_NAME" \
  connection.autoconnect yes
```

The connection profile name might differ from the SSID. --ask avoids putting the Wi-Fi password directly in shell history. If the interface is still missing, inspect USB detection, module logs, and device mode instead of re-running installs blindly.

3.5 Test network and reboot persistence

```
nmcli device status
ping -c 4 1.1.1.1
getent hosts github.com
```

Test reboot **with a local console available**:

```
sudo reboot
```

After reboot:

```
lsmod | grep aic
nmcli device status
ping -c 4 1.1.1.1
```

If the adapter reconnects, manual installation is operational for that kernel. If a future kernel upgrade breaks Wi-Fi, boot a known-good kernel with local console access, rebuild against that new kernel's headers, or adopt a clean DKMS-managed install.

4. DKMS configuration and installation (alternative to Section 3)

DKMS = Dynamic Kernel Module Support. It attempts to rebuild a registered out-of-tree module when a new kernel is installed. DKMS version, driver version, and kernel version are **different** numbers and do not need to match. Rebuilds can still fail if kernel APIs change.

4.1 Decide whether to change the working node

On the controller or any node where manual Wi-Fi already works, do not immediately install DKMS on top of that driver. First inspect what is present:

```
uname -r
dkms status
dpkg -l | grep -Ei 'aic8800|dkms'
modinfo -n aic8800_fdrv
modinfo -F vermagic aic8800_fdrv
sudo find /lib/modules/$(uname -r) -type f \
 \( -name 'aic8800_fdrv.ko*' -o -name 'aic_load_fw.ko*' \)
```

An earlier controller output showed aic8800dc/6.4.3.0-patched.18 (from **Kiborgik**, a *different* repository) with installed (original modules exist) (Differences between built and installed modules). That warning does **not** prove a defective driver; it says DKMS has found mismatching built/installed copies. Keep that node working until you have a local recovery path and a reason to repair it. Don't delete .ko files, run dkms remove, or combine competing drivers without a planned rollback.

4.2 ronnyf DKMS package on a clean node

This is an **alternative installation path**, not a continuation of the manual make install procedure. The ronnyf **release page** documents Debian/Ubuntu package aic8800-fdrv-dkms_6.4.3.0-8_all.deb for its v6.4.3.0-8 release (verified November 6, 2026). Before downloading, confirm that the chosen release still contains the asset.

```
sudo apt update
sudo apt install -y dkms build-essential \
 linux-headers-$(uname -r) curl eject
cd ~
curl -fL --retry 3 \
 -o aic8800-fdrv-dkms_6.4.3.0-8_all.deb \
 https://github.com/ronnyf/AIC8800-Linux-Driver/releases/download/v6.4.3.0-8/aic8800-fdrv-dkms_6.4.3.0-8_all.deb
file aic8800-fdrv-dkms_6.4.3.0-8_all.deb
dpkg-deb --info aic8800-fdrv-dkms_6.4.3.0-8_all.deb
```

Stop here if the file reports ASCII text, is tiny, or dpkg-deb --info fails. A 19-byte or 9 KB text file is not a valid .deb. The wildcard <version> shown in some READMEs is only a placeholder; never type it literally. .pkg.tar.zst packages are for Arch/pacman, **not Ubuntu**.

If and only if the package is valid and there is **no conflicting driver installation**, install:

```
sudo dpkg -i ./aic8800-fdrv-dkms_6.4.3.0-8_all.deb
sudo dkms autoinstall
dkms status
modinfo -n aic8800_fdrv
```

Then run sudo modprobe aic_load_fw, sudo modprobe aic8800_fdrv as needed, complete the **Wi-Fi connection** commands in Section 3.4, and reboot-test from local console. If DKMS compilation fails, use Section 6 to inspect the build log; do not repeatedly overlay the manual driver.

4.3 What an effective DKMS configuration looks like

The ronnyf repository contains a dkms.conf declaring the package, both modules, and auto-install. Its key fields are conceptually:

```
PACKAGE_NAME="aic8800-fdrv-dkms"
PACKAGE_VERSION="6.4.3.0"
BUILT_MODULE_NAME[0]="aic_load_fw"
BUILT_MODULE_NAME[1]="aic8800_fdrv"
AUTOINSTALL="yes"
```

Do not paste this abbreviated example over the real dkms.conf. The complete file also specifies build and installation paths and is installed in the required DKMS source tree by the packaging workflow. The build source can still be incompatible with a future kernel even when AUTOINSTALL="yes".

To examine an existing registration without modifying it:

```
dkms status
sudo find /var/lib/dkms -name make.log
sudo find /var/lib/dkms -type f -name '*.ko*'
```

If the build log exists, read the latest relevant one with sudo tail -n 80 /FULL/PATH/TO/make.log. For a future kernel update, make sure matching headers are installed, then check the DKMS status for the **new** kernel before depending on Wi-Fi.

5. Tailscale: remote access to all three nodes

Tailscale supplies an encrypted private network for your cluster without requiring public inbound port forwarding. **Complete the network and clock steps first.** Install the same Tailscale account/tailnet on the controller, both workers, and the Windows laptop you will use to connect.

5.1 Install on each Ubuntu node

Choose the official install method below; review third-party scripts before executing them if you prefer. Tailscale also offers manual Ubuntu packages.

```
sudo apt install -y curl
curl -fsSL https://tailscale.com/install.sh | sh
sudo systemctl enable --now tailscaled
sudo tailscale up
tailscale status
tailscale ip -4
```

sudo tailscale up prints a sign-in URL. Open it in a browser on another device, authenticate, and confirm the node appears at https://login.tailscale.com/admin/machines. The IP printed by tailscale ip -4 is the node's **private Tailscale IP**, not your public WAN address.

5.2 Choose one SSH approach

A. Traditional OpenSSH over Tailscale (easiest to reason about). It keeps normal Linux SSH keys/password policies and uses Tailscale only as the secure network path:

```
sudo apt install -y openssh-server
sudo systemctl enable --now ssh
sudo systemctl status ssh
```

From your Windows laptop (PowerShell, with Tailscale connected):

```
ssh cluster@100.x.y.z
```

Replace cluster with the **actual Linux username** and 100.x.y.z with the worker's Tailscale IPv4 address. Prefer SSH keys. Restrict access with Tailscale grants/firewall rules; avoid exposing port 22 by router port-forwarding.

B. Tailscale SSH (optional alternative). This uses Tailscale's SSH authentication and tailnet SSH access policies instead of the usual sshd authentication on the Tailscale IP:

```
sudo tailscale set --ssh
tailscale status
```

Tailscale SSH also requires SSH permissions in your tailnet **access controls**. Authorize your identity and the required destination/login username; test from the laptop. To disable later, while retaining another way into the machine:

```
sudo tailscale set --ssh=false
```

Do not conflate these two approaches: sudo tailscale set --ssh enables Tailscale SSH, whereas installing openssh-server enables the ordinary SSH daemon. Tailscale SSH claims port 22 *for tailnet traffic to that machine* when enabled. No public port forwarding is needed for either approach.

5.3 Connectivity and access checks

```
systemctl is-active tailscaled
tailscale status
tailscale ip -4
tailscale ping OTHER_NODE_NAME
```

On the laptop, test each worker one at a time. A tailscale ping failure can mean the device is offline or that tailnet access rules prevent it. Avoid creating broad * permissions simply to make a test work. For unattended servers, consider device key expiry policy carefully: disabling expiry reduces maintenance but weakens security if a machine is compromised.

6. Troubleshooting quick reference

Symptom	Likely cause	Check / safe response
Temporary failure resolving	DNS or connection	ping -c 4 1.1.1.1; getent hosts github.com
SSL certificate is not yet valid	Incorrect clock	date; timedatectl; Chrony in Section 2
InRelease is not valid yet	System clock behind repo timestamps	Set accurate date/time, sync, rerun sudo apt update
Could not get lock ... lock-frontend	apt/unattended-upgrades already running	Inspect process/logs; don't delete lock files
dpkg-deb: unexpected end of file	Bad/missing/non-DEB download	file package.deb; dpkg-deb --info; use verified release asset
pacman: command not found	Using Arch instructions on Ubuntu	Use Ubuntu .deb or source build, not .pkg.tar.zst
Wi-Fi not shown in nmcli	USB not recognized, driver not loaded, wrong mode, or NM not managing	lsusb; dmesg; lsmod; nmcli device status
modprobe: module not found	Module not built/installed for current kernel	uname -r; modinfo; check build errors
DKMS says built/installed modules differ	Duplicate or mismatched module copies	Inspect dkms status, module paths; plan rollback before cleanup
Wi-Fi works, stops after kernel update	Manual module tied to old kernel or failed DKMS rebuild	Boot known-good kernel; check headers and make.log; rebuild carefully
Tailscale connected but SSH denied	Wrong username, missing sshd, key/auth, or policy	tailscale status; systemctl status ssh; tailnet rules

Useful diagnostics to capture when requesting help:

```
uname -r
lsusb
nmcli device status
lsmod | grep aic
dkms status
modinfo -n aic8800_fdrv
sudo dmesg | tail -n 50
```

7. Per-node completion checklist

- ☐ Correct hostname and Ubuntu/kernel noted.
- ☐ lsusb confirms the expected AIC8800 device ID.
- ☐ Chrony is active; Leap status : Normal; timedatectl synchronized.
- ☐ Exactly one AIC8800 driver management method selected (manual or DKMS).
- ☐ Both AIC kernel modules loaded and wireless interface exists.
- ☐ Wi-Fi SSID connected; connection.autoconnect yes on the correct profile.
- ☐ Internet and DNS tests pass.
- ☐ Reboot from local console succeeds and Wi-Fi reconnects.
- ☐ Tailscale is authenticated, online, and visible in the admin console.
- ☐ SSH works from the Windows laptop over Tailscale only as intended.
- ☐ Git commit, kernel version, Tailscale device name, and recovery notes recorded.

8. Primary references (checked November 6, 2026)

1. Ubuntu - Chrony client: https://ubuntu.com/server/docs/how-to/networking/chrony-client/
2. Ubuntu 26.04 release notes - Chrony replaces timesyncd for new installs: https://documentation.ubuntu.com/release-notes/26.04/summary-for-lts-users/
3. ronnyf AIC8800 driver README/manual install and DKMS notes: https://github.com/ronnyf/AIC8800-Linux-Driver
4. ronnyf release v6.4.3.0-8 Ubuntu .deb instructions: https://github.com/ronnyf/AIC8800-Linux-Driver/releases/tag/v6.4.3.0-8
5. ronnyf Makefile and DKMS config: https://github.com/ronnyf/AIC8800-Linux-Driver/blob/main/drivers/aic8800/Makefile and https://github.com/ronnyf/AIC8800-Linux-Driver/blob/main/dkms.conf
6. Kiborgik patched alternative - *different source family*: https://github.com/Kiborgik/aic8800dc-linux-patched
7. Tailscale install for Linux: https://tailscale.com/docs/install/linux
8. Tailscale SSH and access policy: https://tailscale.com/docs/features/tailscale-ssh

Documentation scope: Ubuntu Server/ZimaBlade tutorial. Commands are intended to be executed deliberately, one node at a time; the runbook is not an automated installer.